Student Data Smart Contract: Structure, Array and Fallback

Solidity ^0.8.20 | Remix IDE | Tested on Remix VM, deployed on Ethereum Sepolia test network via MetaMask

Step 1: Smart contract code

```
pragma solidity ^0.8.20;

contract StudentData {

    struct Student {
        uint256 rollNo;
        string name;
        uint8 age;
        string course;
    }

    Student[] public students;

    event StudentAdded(uint256 indexed rollNo, string name);
    event FallbackCalled(address indexed sender, uint256 value, bytes data);

    function addStudent(
        uint256 _rollNo,
        string memory _name,
        uint8 _age,
        string memory _course
    ) public {
        students.push(Student(_rollNo, _name, _age, _course));
        emit StudentAdded(_rollNo, _name);
    }

    function getStudent(uint256 index)
        public
        view
        returns (uint256 rollNo, string memory name, uint8 age, string memory course)
    {
        require(index < students.length, "Student does not exist");
        Student memory s = students[index];
        return (s.rollNo, s.name, s.age, s.course);
    }

    function getTotalStudents() public view returns (uint256) {
        return students.length;
    }

    function getAllStudents() public view returns (Student[] memory) {
        return students;
    }

    receive() external payable {
        emit FallbackCalled(msg.sender, msg.value, "");
    }

    fallback() external payable {
        emit FallbackCalled(msg.sender, msg.value, msg.data);
    }
}
```

StudentData.sol (comments removed)

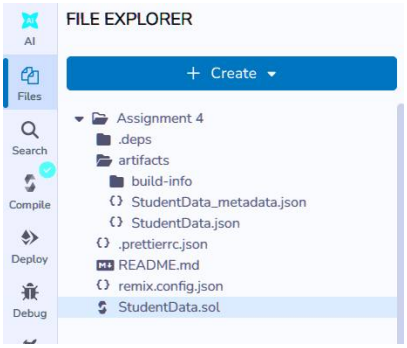


Fig 1: StudentData.sol in the Remix workspace (Assignment 4), compiled

Step 2: Deployed on Remix VM (test run)

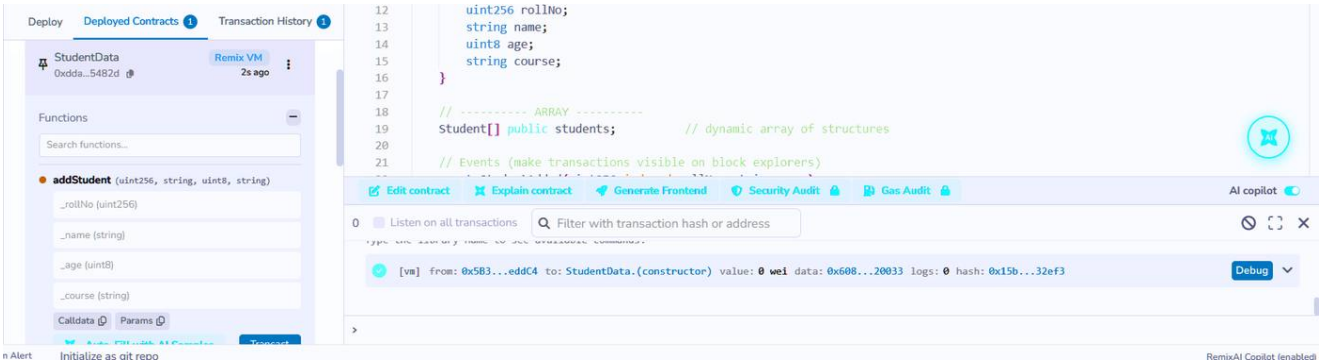


Fig 2: Constructor transaction succeeded on Remix VM

Step 3: Deployment gas on Remix VM



Fig 3: Constructor transaction cost 1006120 gas, execution cost 883514 gas

Step 4: addStudent tested on Remix VM



Fig 4: addStudent(27, Flanker H, 20, Computer Engg) succeeded (green)

Step 5: Fallback function



Fig 5: Low level interaction with calldata 0x1234 runs fallback(); the log shows event FallbackCalled with data 0x1234

Step 6: Fallback and receive transactions

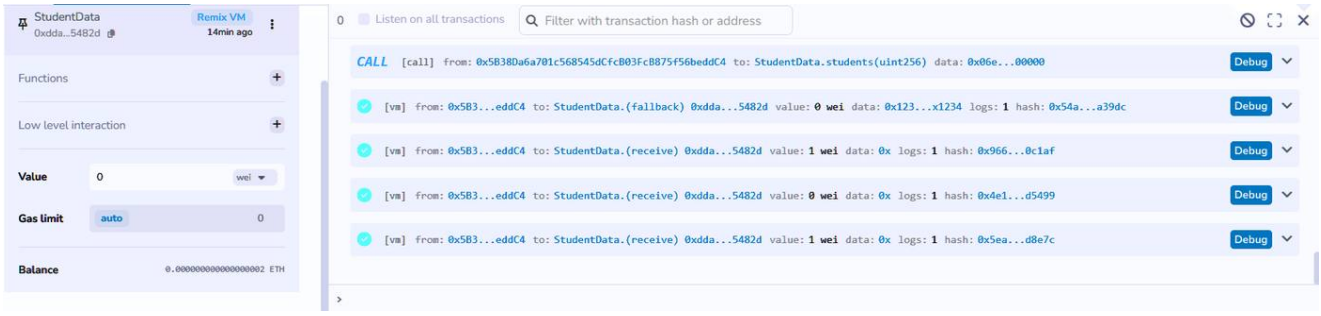


Fig 6: StudentData.(fallback) and StudentData.(receive) calls all succeeded; contract balance is 2 wei

Step 7: Remix connected to Sepolia through MetaMask

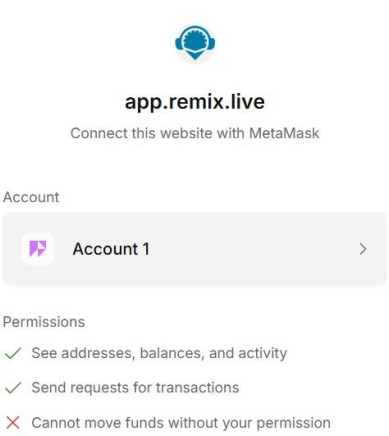


Fig 7: MetaMask asks to connect app.remix.live (Account 1)

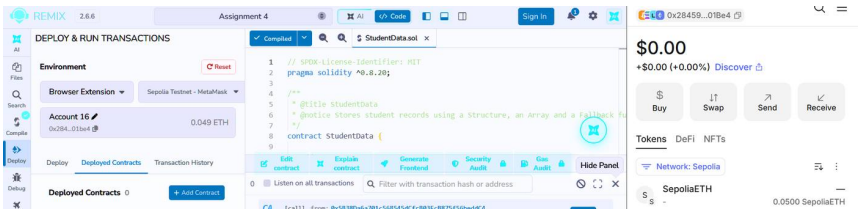


Fig 8: Environment: Browser Extension, Sepolia Testnet - MetaMask; wallet shows about 0.05 SepoliaETH

Step 8: Deploy request on Sepolia



Fig 9: Deploy panel (Sepolia 11155111) and MetaMask Deploy a contract request with network fee

Step 9: Transaction submitted



Fig 10: MetaMask shows Transaction submitted after Confirm

Step 10: Contract deployed on Sepolia

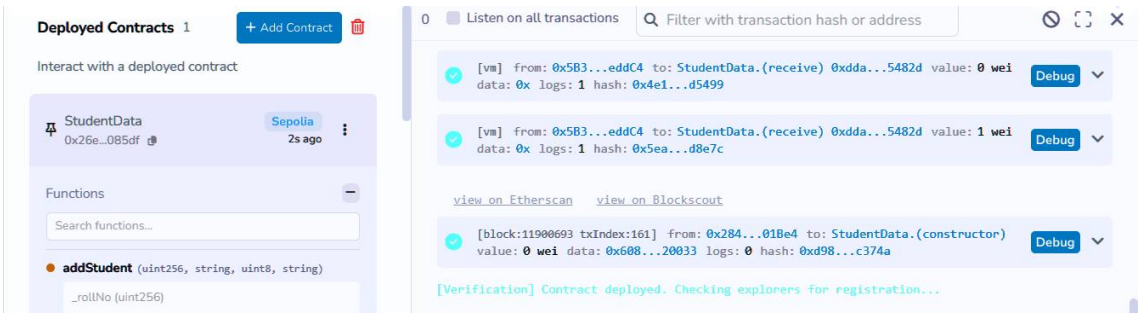


Fig 11: StudentData at 0x26e...085df on Sepolia, constructor in block 11900693

Step 11: addStudent on Sepolia, MetaMask request

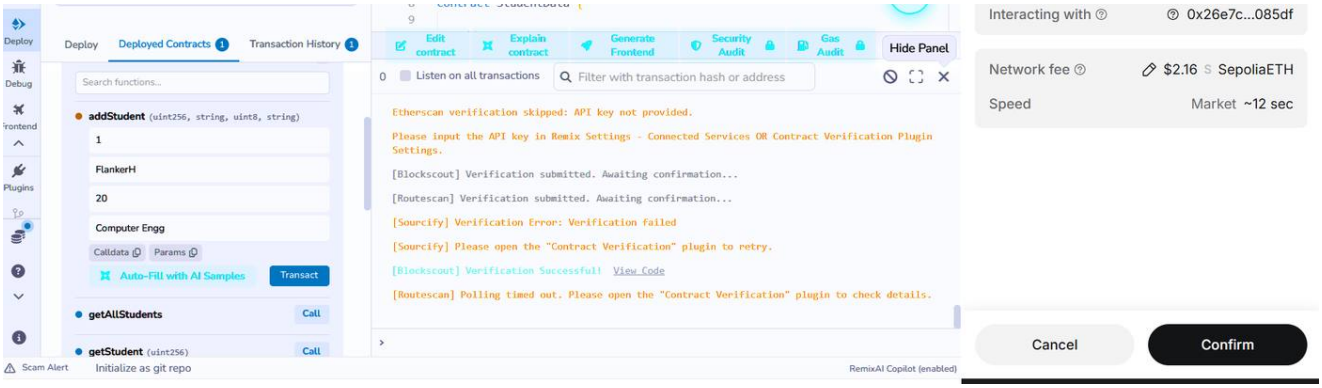


Fig 12: addStudent(1, FlankerH, 20, Computer Engg): MetaMask Transaction request with fee estimate

Step 12: addStudent confirmed



Fig 13: addStudent succeeded on Sepolia, block 11900700, 1 log (StudentAdded event)

Step 13: MetaMask confirmation



Fig 14: MetaMask: Contract interaction, Status Confirmed

Step 14: Reading the stored data (array of structs)

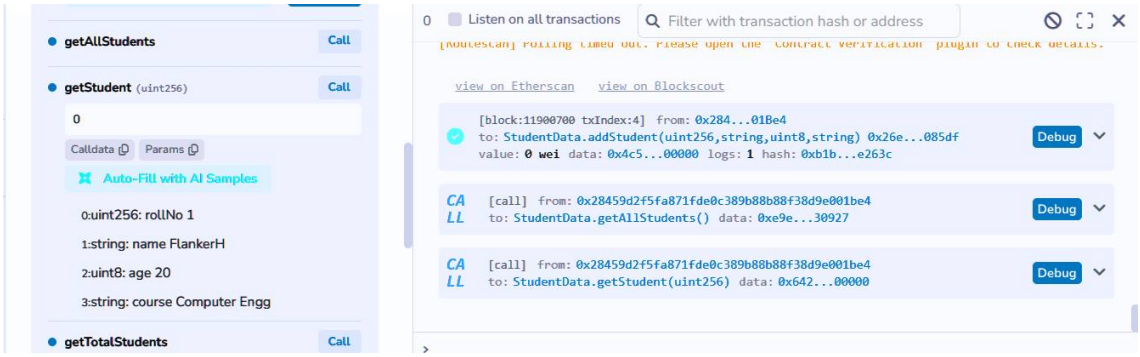


Fig 15: getStudent(0) returns rollNo 1, name FlankerH, age 20, course Computer Engg

Step 15: students(0) public array getter

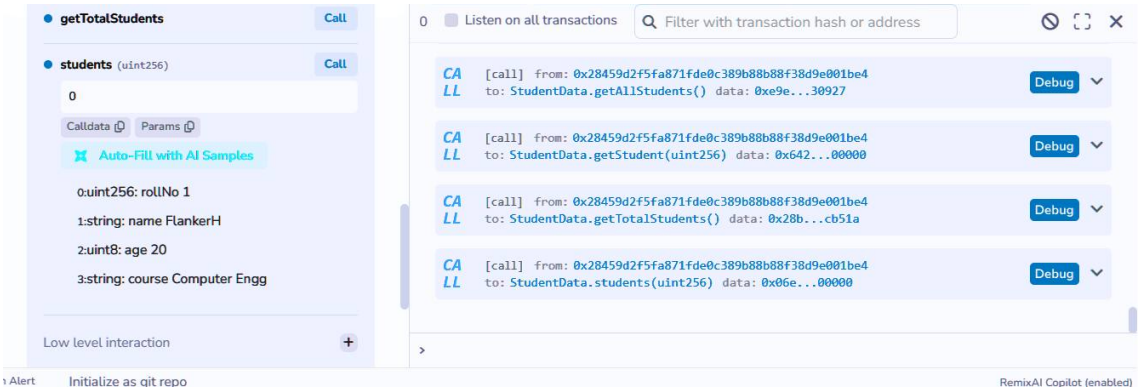


Fig 16: students(0) returns the same record; getAllStudents, getStudent, getTotalStudents calls in the console

Step 16: Transaction fee and gas on Sepolia Etherscan

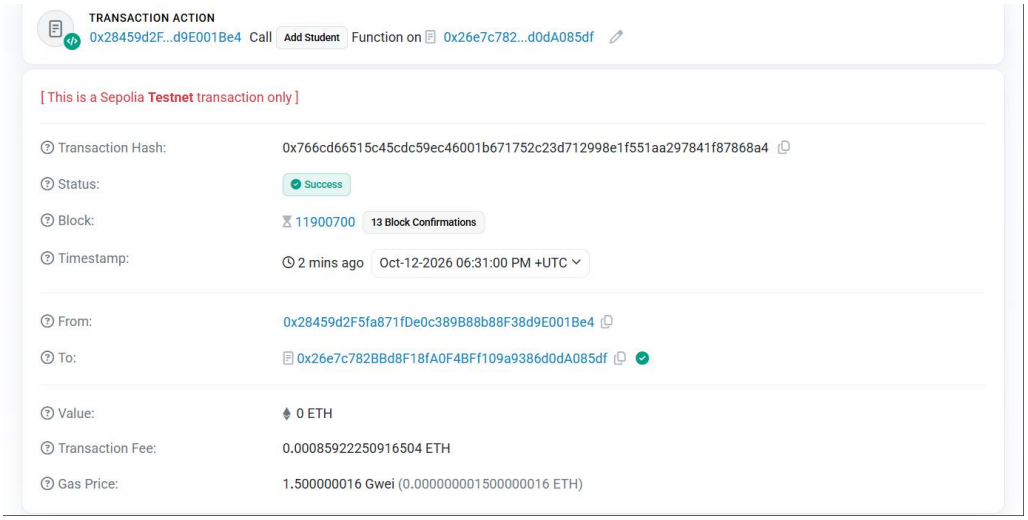


Fig 17: addStudent transaction: Status Success, Transaction Fee 0.00085922250916504 ETH, Gas Price 1.500000016 Gwei

Gas and fee summary

Transaction	Network	Observed
Deploy StudentData	Remix VM	Transaction cost 1,006,120 gas; execution cost 883,514 gas
Deploy StudentData	Sepolia	Success, block 11900693; MetaMask showed a fee estimate (display value only)
addStudent	Sepolia	Fee 0.00085922250916504 ETH at 1.500000016 Gwei, block 11900700
addStudent gas used	Sepolia	572,815 gas (derived: fee / gas price)
fallback (0x1234), receive	Remix VM	Both succeeded; each emitted 1 event log

Conclusion: The contract stores students in a struct inside a dynamic array, and its payable fallback and receive functions accept calls that match no function. Fee = gas used x gas price. Deployment used the most gas (over 1 million on Remix VM), and addStudent used a large amount because it writes to storage.